Lab 6 report

PB24111630 黄雯佩

实验目的与内容

1. 设计完整的流水线CPU
2. 实践冒险处理

逻辑设计

1. 如果存在状态机，请绘制出状态机的状态转换图；
2. 请贴出较为核心的代码设计代码，并加以解释说明。

```
module SegCtrl(
    input  wire        rf_we_ex,    //EX寄存器堆写使能
    input  wire [1:0]  rf_wd_sel_ex,    //EX指令写回数据来源
    input  wire [4:0]  rf_wa_ex,    //EX写回目标寄存器
    input  wire [4:0]  rf_ra0_id,
    input  wire [4:0]  rf_ra1_id,
    input  wire [1:0]  npc_sel_ex,  //EX next PC选择信号，不为0时发生跳转

    output wire        stall_pc,    //PC寄存器暂停
    output wire        stall_if_id, //IF/ID段间寄存器暂停
    output wire        flush_if_id, //清空IF/ID 段间寄存器
    output wire        flush_id_ex  //清空ID/EX 段间寄存器
);

    wire load_use = rf_we_ex && //EX段指令会写回寄存器
                    (rf_wd_sel_ex == 2'b01) &&  //写回数据来自内存（load
                    (rf_wa_ex != 5'd0) &&
                    ((rf_wa_ex == rf_ra0_id) || (rf_wa_ex == rf_ra1_id));   //要写回的数据正

    wire ctrl_hazard = |npc_sel_ex; //npc_sel_ex[1] || npc_sel_ex[0]，不是2'b00就跳转

    assign stall_pc     = load_use && !ctrl_hazard;
    assign stall_if_id  = load_use && !ctrl_hazard;
    assign flush_id_ex  = load_use ? 1'b1 : (ctrl_hazard ? 1'b1 : 1'b0);
    assign flush_if_id  = ctrl_hazard ? 1'b1 : 1'b0;

endmodule
```

SegCtrl模块，解决load-use和控制冒险
实现stall和flush

```
module Fowarding(
    input  wire [4:0]  rf_ra0_ex,   //传入EX段的读寄存器地址
    input  wire [4:0]  rf_ra1_ex,

    input  wire [4:0]  rf_wa_mem,   //写目标寄存器
    input  wire [4:0]  rf_wa_wb,
    input  wire        rf_we_mem,   //MEM段的寄存器堆写使能信号
    input  wire        rf_we_wb,    //WB段的寄存器堆写使能信号

    input  wire [31:0] rf_wd_mem,   //寄存器堆的写数据
    input  wire [31:0] rf_wd_wb,

    output wire        rf_rd0_fe,   //前递使能信号
    output wire        rf_rd1_fe,
    output wire [31:0] rf_rd0_fd,   //前递数据信号 read data
    output wire [31:0] rf_rd1_fd
);

    wire forward_a_mem = rf_we_mem && (rf_wa_mem != 5'd0) && (rf_wa_mem == rf_ra0_ex);
    wire forward_a_wb  = rf_we_wb  && (rf_wa_wb  != 5'd0) && (rf_wa_wb  == rf_ra0_ex) && !forward_a_mem;    //mem阶段的数据更新，
    wire forward_b_mem = rf_we_mem && (rf_wa_mem != 5'd0) && (rf_wa_mem == rf_ra1_ex);
    wire forward_b_wb  = rf_we_wb  && (rf_wa_wb  != 5'd0) && (rf_wa_wb  == rf_ra1_ex) && !forward_b_mem;

    assign rf_rd0_fe = forward_a_mem || forward_a_wb;
    assign rf_rd1_fe = forward_b_mem || forward_b_wb;

    assign rf_rd0_fd = forward_a_mem ? rf_wd_mem : rf_wd_wb;    //前递无效时，默认输出rf_wd_wb
    assign rf_rd1_fd = forward_b_mem ? rf_wd_mem : rf_wd_wb;

endmodule
```

Forwarding模块
解决数据冒险冲突问题，前递无效时，默认输出rf_wd_wb

仿真结果与分析

1. 请给出仿真文件的运行结果截图，并对结果加以阐释；

```
ubuntu@VM13199-hwp:~/COD25-Simulation-Framework$ cmake -B build && cmake --build build
[100%] Built target sim
ubuntu@VM13199-hwp:~/COD25-Simulation-Framework$ ./build/sim
Simulation started...
Difftest level: FULL.
Hit good trap.
Simulation finished.
Time:                   844
Instruction count:      417
PC:                     0x00400680
Instruction:            0x00100073
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x00000001 [ 2] = 0x000006E3 [ 3] = 0xF4B2E614
[ 4] = 0x00000000 [ 5] = 0x00000000 [ 6] = 0x00000004 [ 7] = 0x7F77FFFF
[ 8] = 0xDFFCFFD5 [ 9] = 0x60CBE000 [10] = 0x006FFEFF [11] = 0xF1165000
[12] = 0xF4B2E614 [13] = 0x7594D000 [14] = 0x00000000 [15] = 0x0A438000
[16] = 0x0A438000 [17] = 0x60CBE000 [18] = 0x00000001 [19] = 0x006FFEFF
[20] = 0xB474B650 [21] = 0x00000000 [22] = 0xC2A83000 [23] = 0x9B4AD664
[24] = 0x00000000 [25] = 0xB474B650 [26] = 0xA08B002A [27] = 0x00000004
[28] = 0x97F8E63C [29] = 0xB006F628 [30] = 0x9B4AD664 [31] = 0xB006F628
ubuntu@VM13199-hwp:~/COD25-Simulation-Framework$
```

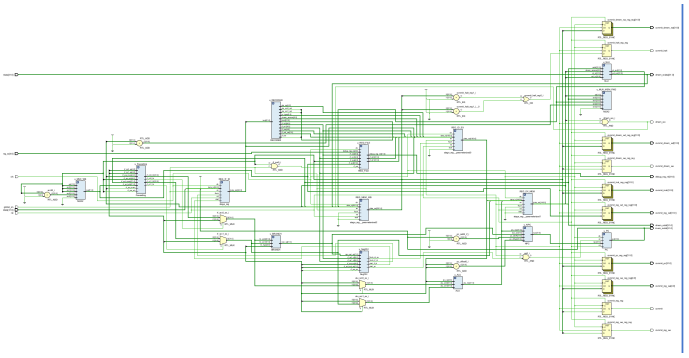
```
ubuntu@VM13199-hwp:~/COD25-Simulation-Framework$ ./build/sim
Simulation started...
Difftest level: FULL.
Hit good trap.
Simulation finished.
Time:                   1226
Instruction count:      492
PC:                     0x004007E8
Instruction:            0x00100073
Registers at the end:
[ 0] = 0x00000000 [ 1] = 0x1002017C [ 2] = 0x00000000 [ 3] = 0x00000000
[ 4] = 0x00000000 [ 5] = 0x10020B53 [ 6] = 0x10015B8D [ 7] = 0x00000000
[ 8] = 0x00000078 [ 9] = 0x00000089 [10] = 0x00000089 [11] = 0x1001C80D
[12] = 0x00000000 [13] = 0x0000003E [14] = 0x10026042 [15] = 0x00000000
[16] = 0x00000000 [17] = 0x00000000 [18] = 0x00000000 [19] = 0x00000000
[20] = 0x10010000 [21] = 0x00000345 [22] = 0x00000345 [23] = 0x00000456
[24] = 0x00000456 [25] = 0x00000111 [26] = 0x00005789 [27] = 0x00005789
[28] = 0x00000000 [29] = 0x00000000 [30] = 0x10011CE8 [31] = 0x0000007D
ubuntu@VM13199-hwp:~/COD25-Simulation-Framework$
```

和要求结果一致

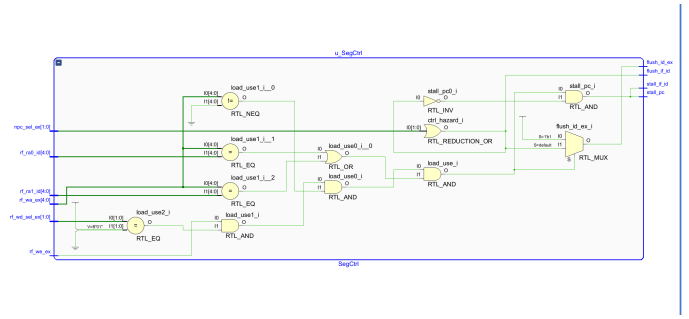
2. 请贴出有特点的仿真测试文件，并说明你在编写仿真测试文件时，对各类情况的考虑（选做）。
   使用课程提供的仿真框架

电路设计与分析

1. 请给出完整的RTL电路图。若某模块较为复杂，也可以再给出该模块的RTL电路图；



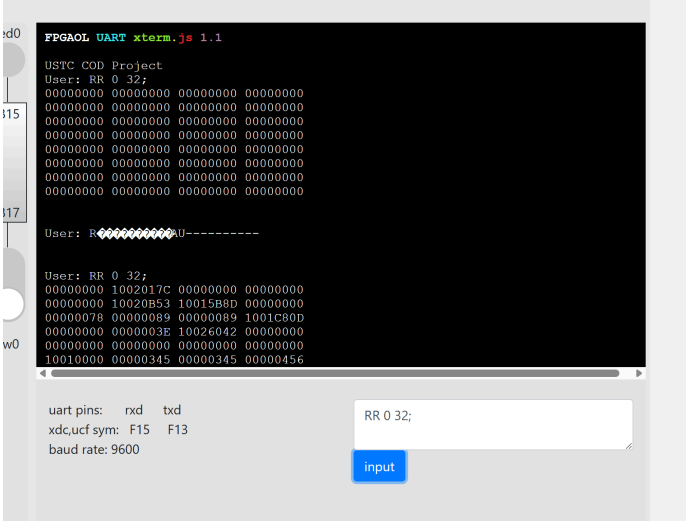
完整电路图



segctrl模块

测试结果与分析

1. 请拍照并附上实验上板结果，以佐证设计的正确性；



完整流水线CPU的上板测试

2. 对实验上板结果进行简要的说明。
   和要求结果一致

总结

1. 请对本次实验中你完成的任务进行简要总结，并总结自己的收获和体验；
   主要设计了forwarding和segctrl模块，解决流水线CPU的冒险冲突
2. 如果对本次实验的设计或助教、老师有建议，可以在这里写下，助教和老师会认真阅读并讨论.